

Могућа и постојећа решења

Наивно решење

Опште решење реализације Медијатор обрасца је приказано сликом Figure 1. Општа реализација обрасца подразумева увођење абстракције (интерфјес *Mediator*) које је асоцирана са конкретном имплементацијом логике Медијатор објекта (*ConcreteMediator*). Приликом позива методе за примање одређеног захтева (npr. *dispatch*), конкретна имплементација обрасца је у стању да констатује којој компоненти је неопходно делегирати даље извршавање (нпр *ConcreteColleague1*).

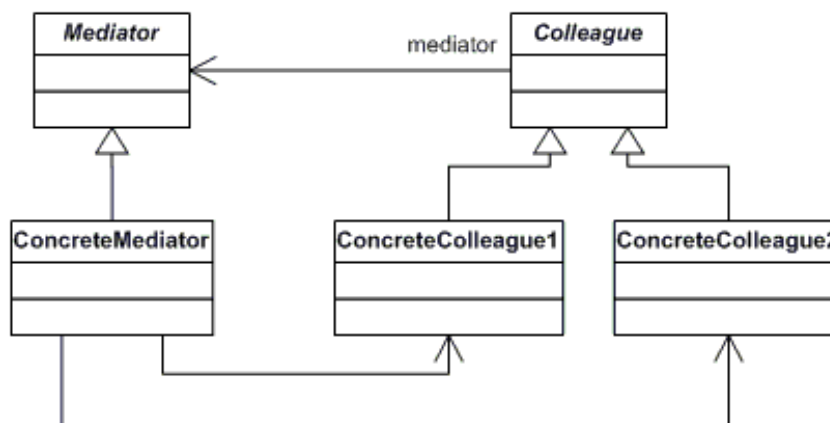


Figure 1: Дијаграм опште имплементације *Mediator* обрасца.

У општем случају, овакво решење је задовољавајуће за једноставније сценарије коришћења. Упркос томе, главни проблеми који се јављају при стварној имплементацији су:

- **Пораст величине посредничког објекта (*God Object Problem*)** – Како систем расте и креирају се нове функционалности, јавља се потреба за проширењем кода посредничког објекта. Ово у најужем смислу обухвата креирање нових *Colleague* објеката и њихову регистрацију. У радним оквирима *Java* екосистема (попут *Spring* радног оквира), ово се манифестује кроз стално инјектовање нових зависности унутар посредничког објекта.
- **Кршење принципа отворености/затворености (*Open/Closed Principle*)** – Свако увођење нове функционалности или међусобне комуникације захтева директну модификацију постојеће класе *ConcreteMediator*, уместо њеног проширивања без измене постојећег кода.
- **Сложеност тестирања** – Због броја зависности које посреднички објекат акумулира, писање јединичних тестова за *ConcreteMediator* постаје компликовано, јер захтева конфигурисање великог броја лажних објеката (*mock* објеката).

Постојећа решења у *Java* екосистему

Потреба за сегрегацијом контролних токова и централизацијом комуникационе логике је у *Java* екосистемима произвела неколико познатих решења.

Axon Framework

Axon Framework је софтверски оквир намењен изградњи дистрибуираних *Java* система. оквир се у великој мери ослања на архитектуру вођену догађајима (*Event-Driven Architecture*) и принцип *Event Sourcing*-а.

У контексту *Mediator* обрасца, *Axon* елиминише проблем монолитног и преоптерећеног посредничког објекта тако што уводи децентрализоване магистрале порука (*Message Buses*).

Уместо једне конкретне класе која зна за све компоненте, систем се заснива на три кључна типа порука, за које постоје специјализовани посредници:

- **Command Bus** – Посредник задужен за рутирање команди (намера за променом стања система) до њихових специфичних обрађивача (@CommandHandler). Свака команда има тачно једног одредишног примаоца, чиме се постиже чиста сегрегација пословне логике.
- **Event Bus** – Након што се стање система измени, генерише се догађај који се путем ове магистрале дистрибуира до свих заинтересованих компоненти (@EventHandler). Ово омогућава лабаво повезивање, јер компонента која је иницирала промену нема никакво сазнање о томе које све компоненте реагују на ту промену.
- **Query Bus** – Магистрала намењена рутирању упита за приказ података до одговарајућих обрађивача (@QueryHandler), подржавајући и концепте попут тачка-до-тачке, раштрканих (*scatter-gather*) или претплатничких упита.

Главна предност коју *Axon* доноси у односу на наивну имплементацију јесте динамичка регистрација компоненти, односно елиминација потребе за ручном регистрацијом и инјекцијом компоненти. Нови обрађивачи команди или догађаја се региструју путем анотација, без икакве потребе за модификацијом самог посредничког објекта. Овиме се поштује *Open/Closed* принцип. Међутим, највећа мана овог оквира је висока комплексност, велика потреба за савладавањем оквира и инфраструктурно оптерећење. Мане *Axon* оквира условљавају да је оквир често превише комплексно решење за стандардне пословне апликације.

PipelinR

PipelinR је *Java* библиотека примарно намењена за *Spring* радни оквир. Библиотека за циљ има да омогући чисту имплементацију команди и упита, без комплексне инфраструктуре какву захтевају већи радни оквири попут *Axon*-а. Дизајнирана је по узору на популарне библиотеке из других екосистема (као што је *MediatR* у *.NET* екосистему).

Уместо централног објекта који садржи референце ка свим сервисима, *PipelinR* уводи концепт цевовода (*pipeline*) и заснива се на три основна градивна блока:

- **Command (или Query)** – Представља једноставан објекат који описује намеру или захтев.
- **Handler** – Компонента задужена за извршавање конкретне пословне логике везане за одређену команду.
- **Pipeline** – Сам посреднички објекат (*Mediator*) који прихвата команду и аутоматски проналази и покреће одговарајући *Handler*.

Библиотека не поседује уграђен механизам за аутоматско скенирање класа. Упркос томе, омогућава да се кроз *Spring* конфигурационе класе прикупе сви обрађивачи (*Command.Handler*) из апликативног контекста и проследе главном *Pipeline* објекту.

Након што се ова почетна конфигурација постави, увођење нове функционалности захтева само креирање нових *Command* и *Handler* класа (означених са анотацијом @Component). *Spring* ће их аутоматски скенирати и доставити цевоводу, чиме се у потпуности елиминише потреба за ручном инјекцијом нових зависности у сам посреднички објекат и задовољава *Open/Closed* принцип.

Главна мана овог решења је што је библиотека ограничена искључиво на синхрону комуникацију по моделу “један на један” (једна команда – један обрађивач), па није погодна

за системе који захтевају асинхроно раштркавање порука или архитектуру вођену догађајима са више претплатника.

Постојећа решења из других екосистема

MediatR у C#

MediatR је једна од најпопуларнијих и најшире прихваћених библиотека у .NET екосистему. Библиотека је постала стандард за имплементацију *Mediator* обрасца и *CQRS* архитектуре у овом окружењу. Својим именом и утицајем библиотека је поставила образац са деривате *Mediator* имплементације у другим програмским језицима.

MediatR се заснива на истом концепту раздвајања порука на команде/упите и њихове обрађиваче. Главни посреднички објекат прихвата поруку и динамички је рутира до одговарајућег обрађивача. Као софтверско решење, он нуди неколико кључних предности у решавању проблема наивне имплементације:

- **Аутоматско скенирање** – За разлику од својих деривата у другим језицима, *MediatR* долази са уграђеним екстензијама за .NET *Dependency Injection* контејнер. То омогућава да се унутар конфигурације апликације позове само једна метода која аутоматски скенира склопове (*assemblies*), проналази све дефинисане обрађиваче и региструје их. Тиме се у потпуности елиминише потреба за модификацијом посредничког објекта и успешно имплементира *Open/Closed* принцип.
- **Разноврсност комуникационих модела** – Поред синхроног “један на један” рутирања, *MediatR* подржава и модел објављивања догађаја вишеструким претплатницима (такозване нотификације), што му омогућава да функционише и као локална магистрала догађаја унутар једне апликације.
- **Цевоводи (*Pipeline Behaviors*)** – Ова функција омогућава креирање пресретаца који омогућавају једноставно убацивање попречних функционалности (*cross-cutting concerns*) попут валидације, логовања, кеширања и управљања трансакцијама дуж саме путање захтева, унапређујући тако сегрегацију пословне логике.

Опште мане имплементација *Mediator* обрасца у различитим екосистемима

Иако наведене имплементације решавају недостатке наивног приступа, оне често отварају нове изазове и импликације архитектуралне природе. Неки од проблема везаних за саму архитектуру система су:

- **Ризик од неисправне резолуције и преклапања обрађивача** – Уколико се грешком региструје више обрађивача за исти тип команде или упита, нарушава се стриктна “један-на-један” веза између поруке и њеног јединственог извршиоца. Оваква лоше постављена конфигурација се не уочава током компајлирања, већ се испољава искључиво у време извршавања апликације (*runtime*). То резултује грешкама у пословној логици или бацањем изузетака. Додатно, неки оквири једноставно пребришу претходне обрађиваче и покрену само последњи регистровани.
- **Нарушавање принципа јединствене одговорности (*Single Responsibility Principle*)** – Класа која има улогу обрађивача порука требало би да буде задужена за извршавање логике једног и само једног типа поруке. Међутим, у већини актуелних библиотека не постоји ограничење високог нивоа које би експлицитно забранило да једна класа бива задужена за обраду различитих порука. Дозвољавањем овакве праксе, класа брзо

акумулира вишеструке одговорности, чиме се директно крши овај фундаментални принцип објектно-оријентисаног дизајна.

- **Проблем наслеђивања између класа порука** – Уколико један тип поруке (класа) наслеђује други, семантика и логика извршавања постају нејасне. Са становишта архитектуре, поставља се комплексно питање резолуције: да ли поруку изведеног (подређеног) типа треба обрадити искључиво путем обрађивача који је за њу експлицитно дефинисан, или је треба проследити и обрађивачу надтипа, или пак активирати оба у одређеном редоследу. Библиотеке често немају једнообразан одговор на овај проблем, што уноси непредвидивост у рад система.
- **Заобилажење посредничког објекта директном инјекцијом компоненти** – Дизајн заснован на Медијатору захтева да се међусобна интеракција између зависних компоненти спроводи искључиво преко њега. Међутим, у пракси је и даље технички дозвољено да се инстанцирају или директно инјектују друге компоненте унутар једног обрађивача. Експлицитно повезивање компоненти чијом би комуникацијом примарно требало да руководи посредник нарушава основни смисао његовог постојања и архитектуре лабавог повезивања.
- **Непостојање обрађивача за конкретни тип поруке** – Већина имплементација омогућава да се порука пошаље кроз систем чак и када не постоји регистрован обрађивач за њу. Понашање система варира од оквира до оквира: поједине библиотеке ће бацити изузетак и прекинути извршавања, док ће друге једноставно “прогутати” поруку без икаквог обавештења. Креирање објеката порука који немају коњунктивни пар у виду обрађивача је бесмислено.